

Activity No. 7.1	
Sorting Algorithms	
Course Code: CPE010	Program: Computer Engineering
Course Title: Data Structures and Algorithms	Date Performed:16/10/2024
Section:CPE21S4	Date Submitted: 16/10/2024
Name(s):Jimenez, Christian Joros R.	Instructor: Engr. Sayo
6. Output	

Code + Console Screenshot	<pre> #include <iostream> #include <cstdlib> #include <algorithm> using namespace std; void createRandomArray(int arr[], int size) { srand(time(0)); for (int i = 0; i < size; ++i) { arr[i] = rand() % 100; } } void printArray(int arr[], int size) { for (int i = 0; i < size; ++i) { cout << arr[i] << " "; } cout << std::endl; } int main() { const int size = 100; int arr[size]; // Create random array createRandomArray(arr, size); cout << "Original Array: "; printArray(arr, size); return 0; } </pre> Output: <pre> Original Array: 76 36 78 9 17 15 32 0 46 8 57 21 8 70 28 33 80 79 22 43 46 56 33 65 1 8 85 11 21 86 83 78 14 71 56 23 88 72 55 40 70 16 97 91 24 19 19 10 0 50 32 95 49 41 80 14 59 18 77 33 4 13 11 18 84 68 93 25 92 48 65 62 16 15 54 41 86 25 51 38 28 35 34 77 76 66 91 88 84 21 21 88 34 32 58 18 52 3 43 96 4 </pre>
---------------------------	---

Observation	The code showed 100 integer values in a randomly generated order.

Table 7-1. Array of Values for Sort Algorithm
Testing

Code + Console Screenshot

```
#include <iostream>
using namespace std;

template <typename T>
void bubbleSort(T arr[], size_t
arrSize) { for ( int i = 0; i <
arrSize; i++) {
    for (int j = i + 1; j <
arrSize; j++) { if (arr[j] >
arr[i]) {swap(arr[j], arr[i]);
    }
    }
}
}

int main() {
    int arr[] = {56, 109, 4, 123, 984, 40, 41};
    size_t arrSize = sizeof(arr) / sizeof(arr[0]);

    std::cout << "Original
array: "; for (int i = 0; i <
arrSize; i++) {cout <<
arr[i] << " ";
    }
    cout << endl; bubbleSort(arr,
arrSize);

    cout << "Sorted array: ";
    for (int i = 0; i < arrSize;
i++) {cout << arr[i] << " ";
    }
    cout << endl;

    return 0;
}
```

Output:

```
Original array: 56 109 4 123 984 40 41
Sorted array: 984 123 109 56 41 40 4
```

Observation

Iteratively going through the list, comparing adjacent elements, and switching them if they are out of order is how the bubble sort technique works.

Table 7-2. Bubble Sort Technique

Code + Console Screenshot

```
#include <iostream>
using namespace std;

template <typename T>
int Routine_Smallest(T arr[], int K, const int
arrSize) { int position = K;
T smallestElem = arr[K];

for (int j = K + 1; j <
arrSize; j++) { if (arr[j] <
smallestElem) {
smallestElem = arr[j];
position = j;
}
}
return position;
}

template <typename T>
void selectionSort(T arr[], const
int N) { for (int i = 0; i < N - 1;
i++) {
int POS = Routine_Smallest(arr, i, N);
std::swap(arr[i], arr[POS]);
}
}

int main() {
int arr[] = {5, 2, 8, 3, 1, 6, 4};
const int N = sizeof(arr) / sizeof(arr[0]);

cout << "Original array: ";
for (int i = 0; i < N; i++)
{cout << arr[i] << " ";
}
cout << endl; selectionSort(arr,

N);

cout << "Sorted array: ";
for (int i = 0; i < N; i++) {
cout << arr[i] << " ";
}
cout << endl;

return 0;
}
```

Output:

```
Original array: 5 2 8 3 1 6 4  
Sorted array: 1 2 3 4 5 6 8
```

Observation

Finding the smallest number in the array's unsorted section and replacing it with the first number there is how the selection sort function operates. Until the entire array is sorted, it repeatedly performs this action. To determine the smallest number, it makes use of a helper function named `Routine_Smallest`.

Table 7-3. Selection Sort Algorithm

Code + Console Screenshot

```
#include <iostream>

using namespace std;

template <typename T>
void insertionSort(T arr[], const int N) {
    for (int K = 1; K < N; K++) {
        T temp = arr[K];
        int J = K - 1;
        while (J >= 0 && arr[J] > temp) {
            arr[J + 1] = arr[J];
            J--;
        }
        arr[J + 1] = temp;
    }
}

int main() {
    int arr[] = {65, 231, 215, 756, 214, 67, 21};
    const int N = sizeof(arr) / sizeof(arr[0]);
    cout << "Original array: ";
    for (int i = 0; i < N; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    insertionSort(arr, N);
    cout << "Sorted array: ";
    for (int i = 0; i < N; i++) {
        cout << arr[i] << " ";
    }
    cout << endl;
    return 0;
}
```

	Original array: 65 231 215 756 214 67 21 Sorted array: 21 65 67 214 215 231 756
--	--

Observations	In essence, the insertion sort function traverses the array one element at a time. It takes each element and places it in the appropriate location within the array's already-sorted section.
--------------	---

--	--

Table 7-4. Insertion Sort Algorithm

7. Supplementary Activity

Pseudocode

Procedure CountVotes()

// Get Votes:

Make a list called "votes" with 101 spaces. Each space will get a random number between 1 and 5.

// Sort the Votes:

Go through the list "votes" and arrange the numbers in order, smallest to biggest.

// Count Votes:

Make a new list called "candidateCounts". Each space in this list will represent a candidate.

Start with the first vote in "votes". Keep track of how many times that same vote number appears in a row.

When you find a different vote number, write down the candidate number and the number of times it appeared in the "candidateCounts" list.

// Find the Winner:

Look at the "candidateCounts" list. Find the candidate with the most votes.

// Print Results:

Show the original votes in the "votes" list.

Say "The winning candidate is Candidate ..." and then say the winning candidate number and how many votes they got.

End Procedure

Code + Screenshot Output

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <random>

using namespace std;

int main() {
    random_device rd;
    mt19937 gen(rd());
    uniform_int_distribution<> distrib(1, 5);
    vector<int> votes(101);
    generate(votes.begin(), votes.end(),
[&distrib, &gen]() { return distrib(gen); });

    for (int i = 1; i < votes.size(); ++i) {
        int key = votes[i];
        int j = i - 1;
        while (j >= 0 && key < votes[j]) {
            votes[j + 1] = votes[j];
            --j;
        }
        votes[j + 1] = key;
    }

    int currentCandidate = votes[0];
    int count = 1;
    vector<pair<int, int>> candidateCounts;
    for (int i = 1; i < votes.size(); ++i) {
        if (votes[i] == currentCandidate) {
            ++count;
        } else {

            candidateCounts.push_back(make_pair(c
urrentCandidate, count));
            currentCandidate = votes[i];
            count = 1;
        }
    }

    candidateCounts.push_back(make_pair(c
urrentCandidate, count));

    int winningCandidate = 0;
    int maxVotes = 0;
    for (const auto& pair : candidateCounts) {
```

```

        if (pair.second > maxVotes) {
            winningCandidate = pair.first;
            maxVotes = pair.second;
        }
    }

    cout << "Unsorted Votes: ";
    for (int vote : votes) {
        cout << vote << " ";
    }

    cout << endl;
    cout << "The winning candidate is  

    Candidate " << winningCandidate  

        << " with " << maxVotes << " votes."  

    << endl;

    return 0;
}

```

```
Unsorted Votes: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
2 2 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 3 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4 4
4 4 4 4 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5
The winning candidate is Candidate 4 with 25 votes.
```

Answer	Question: Justify why you chose to use this sorting algorithm. Since there are only 101 elements in the array of votes (0-100), I decided to use the insertion sort algorithm. Because insertion sort has a time complexity of $O(n)$ in the best case and $O(n^2)$ in the worst, it is typically efficient for such small datasets. Insertion sort is appropriate for this problem, where clarity and ease of use are the main concerns, because it is comparatively easy to comprehend and apply. Question: was your voting counting algorithm effective? Why or why not? I think my voting algorithm is effective because it was able to successfully solve the problem without any problem. It determines the candidate with the most votes by accurately tallying them. The code is also very simple, therefore making it easier to comprehend
---------------	---

8. Conclusion
To sum up, I discovered that sorting strategies are crucial to data structures because they help organize data so that it can be swiftly and effectively searched through. I had the opportunity to investigate various sorting algorithms, such as bubble sort, selection sort, and insertion sort; each has advantages and disadvantages of its own. I discovered how helpful these methods can be when I used them to solve a real-world issue, such as tallying votes and determining the victorious candidate. I experimented with different sorting techniques to see which one interests me the most and used it for the supplementary activity however, all sorting techniques are important and they each have their own advantages and disadvantages that one must consider before using them. I think I did good for this laboratory activity, I did not have a lot of problems completing this activity.
9. Assessment Rubric

Output Console Showing Sorted Array	Manual Count	Count Result of Algorithm
<pre>Unsorted Votes: 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 3 4 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 5 The winning candidate is Candidate 4 with 25 votes.</pre>	Candidate 1: 15 votes Candidate 2: 18 votes Candidate 3: 15 votes Candidate 4: 17 votes Candidate 5: 16 votes	The winning candidate is Candidate 4 with 25 votes.